

面向事件驱动智能体操作系统的跨层能力治理方法

摘要

事件驱动智能体操作系统将外部事件转化为自主任务和工具副作用，但事件来源、任务创建与工具执行分属不同信任域，协议格式校验或单点动作过滤无法证明一次副作用获得了完整链路授权。本文提出跨层能力治理方法 E2AG，将事件声明权、任务创建权和工具副作用权组织为同一任务作用域能力生命周期。E2AG 通过来源-类型契约和目标策略完成任务准入，为获准任务签发对象绑定的短期能力，并在实际 MCP 调用抵达上游前实施第二次强制验证；统一执行证据用于检查授权依赖和定位治理失败。本文给出跨层授权状态模型、完整中介安全性质及其证明概要，并在事件驱动智能体操作系统原型上实现该方法。基于经盲表复核的冻结治理矩阵、合成压力集、真实持久化端到端路径、既有模型工具决策回放、自有设施治理一致性回归和并发故障注入的评估表明：完整方法在保持正常事件可用性的同时正确处理冻结矩阵中的全部违规事件；移除任一执行点都会重新引入与其可见上下文对应的未授权副作用。结果说明，跨层对象绑定与双执行点完整中介能够把模型的不确定决策约束在可验证的系统授权边界内。

关键词：人工智能操作系统；事件驱动智能体；能力治理；完整中介；执行溯源

中图法分类号：TP309

A Cross-Layer Capability Governance Method for Event-Driven Agent Operating Systems

Abstract: Event-driven agent operating systems transform external events into autonomous tasks and tool side effects. Event provenance, task creation, and tool execution nevertheless belong to distinct trust domains, so protocol validation or a single action filter cannot establish that a side effect is authorized by the complete execution chain. This paper presents E2AG, a cross-layer capability-governance method that organizes event-declaration, task-creation, and tool-side-effect authorities into a task-scoped capability lifecycle. E2AG admits tasks through source-type contracts and target policies, issues an object-bound short-lived capability for each admitted task, and enforces the capability again before an actual MCP call reaches its upstream tool. Unified execution evidence checks authorization dependencies and localizes governance failures. We define a cross-layer authorization transition model and a complete-mediation safety property with a proof sketch, and implement E2AG in an event-driven agent-OS prototype. Evaluation with a blindly reviewed frozen governance matrix, a synthetic stress suite, persistent end-to-end executions, replay of previously frozen model tool decisions, a governance-consistency regression on self-operated facilities, and concurrent fault injection shows that the complete method correctly handles every policy-violating event in the frozen matrix while preserving benign availability; removing either enforcement point reintroduces the unauthorized side effects associated with the context visible at that point. The results indicate that cross-layer object binding and dual-point complete mediation can constrain nondeterministic model decisions within verifiable system authorization boundaries.

Key words: agent operating system; event-driven agent; capability governance; complete mediation; execution provenance

引言

在事件驱动智能体操作系统（Agent OS/AIOS）中，Git 推送、邮件、监报告警和业务回调不再只是被动输入，而可以直接创建智能体任务。任务运行时读取事件载荷，调用模型选择工具，再由 MCP

或其他连接器访问代码仓库、数据库、消息系统和设备。与传统请求-响应应用相比，这条链同时包含不可信事件输入、非确定性模型决策和可产生外部副作用的工具调用；治理问题因而不只是“模型选择了什么动作”，而是“该动作能否沿系统路径获得执行权”。

考虑一个具有合法 CloudEvent 结构的 Git 推送事件。不可信来源可能给出与其身份不一致的来源字段，也可能使任务在创建后选择 grant 集合之外的工具。入口模式校验只能确认字段存在，目标智能体的静态工具配置不能表达“本次事件是否有权创建该任务”，而调度时已经允许的任务也可能在运行中选择另一工具。若工具网关只看到工具名而看不到触发事件和任务身份，它同样无法判断一个表面合法的调用是否具有完整授权。上述失效并非同一规则的重复遗漏，而是授权对象在事件、任务和工具三个信任域之间发生了变化。

CloudEvents、A2A 和 MCP 分别规定事件、任务和工具调用对象 [16, 17, 18]，但协议互操作不自动产生跨对象授权。间接提示词注入评测揭示了不可信内容改变工具决策的风险 [6, 7, 8, 9]；AgentSpec 和 CaMeL 等工作把确定性约束置于模型之外 [10, 11]。这些机制分别保护事件格式、运行时动作或数据流，却没有共同回答三个问题：谁有权声明某类事件，该事件能否创建指定智能体任务，以及任务实际选择的工具能否产生副作用。经典参考监控器要求安全相关访问受到不可绕过的完整中介 [20, 21]；在事件驱动 AIOS 中，单个执行点无法同时观察三个对象，完整中介必须跨层组织。

本文把上述问题抽象为跨层授权传播：系统接收不可信事件后，应当逐步收缩而不是隐式扩大权限。事件来源只能声明契约允许的类型；获准事件只能创建目标策略允许的任务；任务只能使用绑定到其事件、智能体、MCP 服务和工具集合的短期能力。任何到达上游的副作用，都应存在可验证的事件-任务-能力-调用依赖。该目标还要求审批、能力过期和事件重放采用单调、原子的状态转移，否则正确的策略判定仍可能被并发竞态破坏。

基于这一抽象，本文提出 E2AG (Event-to-Agent Governance)。E2AG 由治理平面、执行平面和证据平面组成：治理平面拥有来源-类型契约、目标策略、审批和任务能力状态；执行平面在任务创建前与工具调用前设置两个不可绕过的 PEP；证据平面把两个执行点及其对象依赖写入同一 trace。E2AG 不检测自然语言是否恶意，而是在模型之外决定一次任务和一次副作用是否具有完整的系统授权。

本文采用状态迁移系统描述事件接入、任务创建、能力签发与工具调用，给出端到端副作用安全性质和双执行点完整中介命题。该形式化不是装饰性符号：任务接入性质由 $\text{Contract} \times \text{Policy}$ 消融检验，副作用安全性质由固定模型决策的配对执行检验，证据连续性和状态单调性分别由故障注入与并发竞态检验。

本文的主要贡献如下：

1. 提出事件驱动 AIOS 的跨层授权问题，统一刻画事件声明权、任务创建权与工具副作用权，给出可由算法和实验共同检查的安全性质；
2. 设计 E2AG 三平面架构、任务作用域能力和双执行点完整中介算法，使工具调用不能脱离触发它的事件与任务单独获得权限；
3. 在真实事件驱动智能体操作系统原型中实现 E2AG，并通过完整消融、持久化端到端执行、既有模型决策配对回放、故障注入和并发状态实验验证各机制的独立作用与组合效果；
4. 公开区分已验证范围与外部有效性边界：当前证据支持单一 AIOS 原型上的副作用可达性与治理状态性质，不外推为任意提示词注入检测、所有工具传输覆盖或跨 AIOS 通用性。

本文其余部分组织如下：第 1 节介绍相关协议、智能体运行时防护与访问控制基础；第 2 节给出系统模型、不可信输入能力和安全目标；第 3 节介绍 E2AG 总体架构；第 4 节给出跨层治理机制、算法与性质分析；第 5 节说明原型实现；第 6 节报告实验；第 7 节讨论适用边界与有效性威胁；第 8 节总结全文。

1 背景与相关工作

1.1 智能体风险与评测

间接提示词注入源于智能体把不可信数据同时解释为内容和指令,因而可能改变模型行为。InjecAgent 以工具集成任务评估间接注入, AgentDojo 提供包含任务、工具和注入情形的动态环境, ToolEmu 则以模型模拟工具执行结果以扩大长尾风险测试范围 [7, 8, 9]。国内研究从模型自身安全、生成内容安全、隐私泄露和智能体可信等角度形成了风险分类与缓解框架 [3, 4, 5]。这些工作主要观测模型是否产生违规工具决策; E2AG 不判定自然语言意图类别, 而是约束该决策在真实系统执行链中的可达性。

1.2 智能体运行时防护

AgentSpec 通过领域专用规则描述触发条件、谓词和动作, 在智能体运行时强制执行动作规则 [10]。CaMeL 从可信查询提取控制流和数据流, 并用能力约束不可信数据对程序流与敏感数据外泄的影响 [11]。二者说明安全决策需要位于模型之外的确定性系统层。新近工作进一步把授权置于工具调用或智能体运行时: OAP 在调用前执行声明式策略并生成签名审计记录 [12]; ToolGuardian 结合工具准入表征与任务感知的声明式授权 [13]; CASA 将确定性消息约束与任务-工具语义匹配结合 [14]; Agent libOS 则以显式能力中介进程、工具、资源和审批原语 [15]。这些方法共同证明了调用前授权的重要性, 但其授权起点主要是用户任务、运行时进程或待调用工具。E2AG 进一步处理 Connector 外部事件的声明权, 并把事件、系统任务和调用能力绑定为同一生命周期。对已授权工具的参数级数据流约束仍应由 CaMeL 类机制或参数谓词补充。

1.3 协议、参考监控器与能力系统

CloudEvents、A2A 与 MCP 分别定义事件、任务和工具调用对象 [16, 17, 18], 但格式互操作不等于授权传播。PDP/PEP 分离支持在执行点实施结构化策略 [19]; 参考监控器和完整中介原则要求所有安全相关访问经过不可绕过且足够小的检查机制 [20, 21]; 能力系统把访问权封装为可传递、可收缩的对象 [22, 24, 23]。W3C Trace Context 解决跨服务标识传播, 哈希链接和 in-toto 用于验证过程记录的连续性与步骤依赖 [25, 26, 27]。E2AG 的贡献不是重新定义这些协议或密码结构, 而是把它们组织为事件-任务-工具三层的授权传播和完整中介。

表 1 按保护对象、执行点和证据类型比较代表性方法。协议标准不承担治理, 现有运行时方法集中于动作、任务意图或数据流; E2AG 同时约束事件声明、系统任务创建和工具副作用, 并把三个判定关联到同一执行证据。由于这些方法的输入主体和保护对象不同, 本文只进行机制范围比较, 数值实验采用共享代码路径的治理位置基准, 避免把不同系统的公开结果直接排序。

2 系统模型与问题定义

2.1 问题胶囊与信任边界

本文研究如下系统: 多个外部事件源经 Connector 接入 AIOS, AIOS 创建智能体任务, 模型在任务中选择工具, 工具调用对外部资源产生副作用。外部事件源及其载荷不可信; AIOS 治理代码、策略库和持久化状态属于可信计算基; 模型输出不作为授权依据; 外部工具只信任经过调用 PEP 的请求。被保护资产是任务创建权、工具凭据和外部副作用。不可信来源可能提交不一致事件字段、重复事件, 或通过载荷改变模型工具选择; 本文假设其不能直接修改治理代码、策略和数据库。目标是在不分析自然语言意图类别的前提下, 使每个上游副作用都可追溯到合法事件、获准任务和有效任务能力。参数级数据流、治理主机失陷和外部存储整体回滚不在本文范围内。

表 1: 代表性方法的治理范围比较

方法	事件来源绑定	任务创建门控	工具调用强制	跨层执行证据
CloudEvents	—	—	—	—
AgentSpec	—	—	✓	局部规则
CaMeL	—	—	✓	控制/数据流
OAP	—	—	✓	签名调用记录
ToolGuardian	—	—	✓	策略判定
CASA	—	—	✓	任务-工具匹配
Agent libOS	—	—	✓	进程/能力记录
E2AG	✓	✓	✓	✓

设来源主体集合、智能体集合和工具服务集合分别为 $\mathcal{S}$ 、 $\mathcal{A}$ 和 $\mathcal{M}$ 。一次执行涉及事件 e 、任务 t 、任务能力 g 、工具调用 c 与外部副作用 o 。系统状态写为

$$\Sigma = \langle E, T, G, Q, L \rangle, \quad (1)$$

其中 E, T, G, Q, L 分别保存事件、任务、能力、审批状态和执行证据。该状态直接对应第 5 节原型中的 EventLog、A2ATask、ToolGrant、Approval 和 AuditEntry，而不是仅用于记号说明。

事件接入域、智能体执行域和工具副作用域构成两个信任边界：TB1 位于外部来源与 AIOS 之间，TB2 位于 AIOS 与外部工具之间。跨越 TB1 的结构化字段不自动获得事件类型的声明权；跨越 TB2 的模型输出不自动获得工具执行权。

2.2 跨层授权状态迁移

令 $B_C(e)$ 表示事件来源与类型满足某个版本化 Connector 契约， $D_P(e, a) \in \{\text{allow}, \text{deny}, \text{approval}\}$ 表示目标智能体策略判定。审批状态 q 从 pending 只能单次转移到 approved、rejected 或 expired；能力状态从 active 只能转移到 revoked 或 expired。允许产生副作用的状态迁移为

$$\Sigma_0 \xrightarrow{\text{ingest}(e)} \Sigma_1 \xrightarrow{\text{admit}(e, a)} \Sigma_2 \xrightarrow{\text{create}(t)} \Sigma_3 \xrightarrow{\text{issue}(g)} \Sigma_4 \xrightarrow{\text{invoke}(c)} \Sigma_5 \xrightarrow{\text{effect}(o)} \Sigma_6. \quad (2)$$

任一前置条件失败都会进入拒绝、待审批或终止状态，不能跳过相应转换。第 4 节算法实现这些转换，第 6 节分别通过准入消融、双 PEP 配对执行和并发状态实验检查其前置条件与单调性。

2.3 端到端安全性质

能力 g 绑定 trace、事件、任务、智能体、MCP 服务、工具集合、期限和状态。定义

$$\text{Admit}(e, a) \triangleq B_C(e) \wedge (D_P(e, a) = \text{allow} \vee \text{Approved}(e, a)), \quad (3)$$

$$\text{ValidGrant}(g, e, t, a, m, \tau) \triangleq \text{active}(g, \tau) \wedge \text{bind}(g, e, t, a, m). \quad (4)$$

本文要求系统满足以下可检查性质。

- I1. 任务来源闭包：若创建 t ，则存在 e, a 使 $\text{Admit}(e, a)$ 成立且 t 绑定 e, a ；
- I2. 能力作用域闭包：若签发 g ，则它只绑定一个获准任务及其智能体、MCP 服务和允许工具集合，且状态转移单调；
- I3. 副作用安全：若调用 c 产生外部副作用，则必存在 e, t, g, a, m 使

$$\text{Effect}(c) \Rightarrow \text{Admit}(e, a) \wedge \text{ValidGrant}(g, e, t, a, m, \tau) \wedge \text{tool}(c) \in U(g) \wedge e \prec t \prec g \prec c; \quad (5)$$

14. 证据连续性：对同一 trace 的证据序列 $L_p = \langle l_1, \dots, l_n \rangle$ ，序号连续、对象依赖满足状态迁移顺序，且每项前驱哈希等于上一项内容哈希。

式 (5) 是本文的中心安全目标：它把来源、任务、能力和调用置于同一蕴含关系，而不是分别报告若干局部规则命中率。

3 E2AG 总体架构

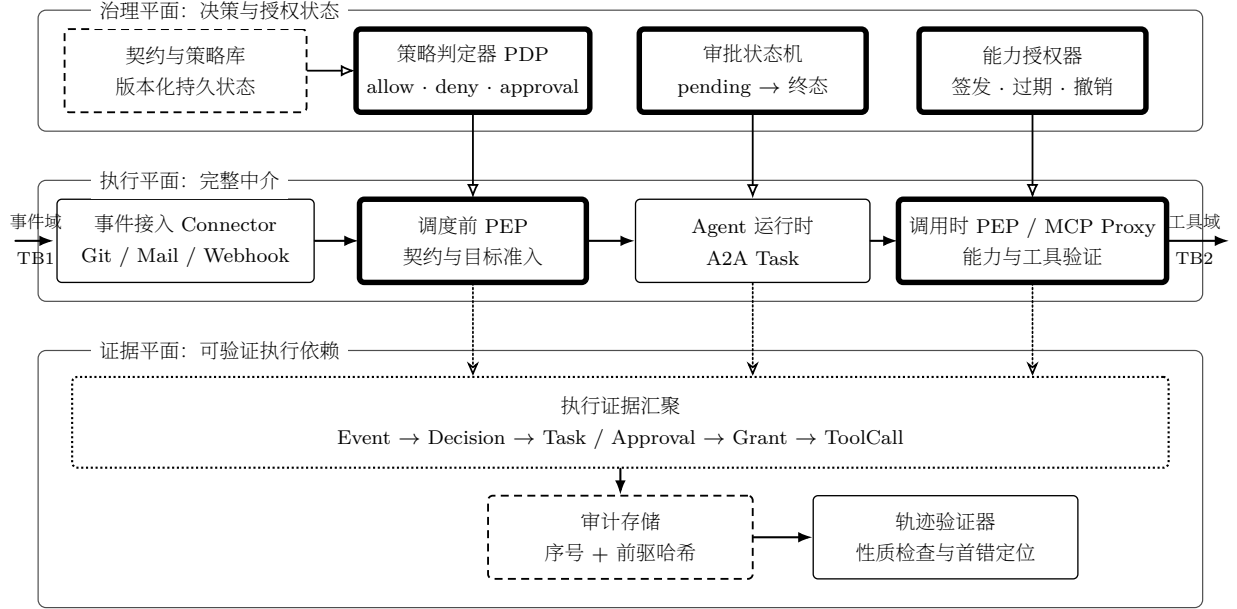


图 1: E2AG 总体架构。三个等宽平面按对象列对齐；实心箭头、空心三角箭头和点线空心燕尾箭头分别表示执行数据、治理控制和证据写入，双线框和虚线框分别表示 E2AG 强制组件和持久状态。

图 1 给出 E2AG 的静态系统架构。治理平面拥有授权决策及其持久状态，执行平面只消费治理结果并实施完整中介，证据平面接收两个执行点和中间对象的状态变化。三个平面共享 trace 和对象标识，但职责不同：PDP 计算策略，PEP 阻止未获准状态迁移，证据平面证明一次决策依赖哪些对象。

3.1 治理平面

契约与策略库存储来源-类型绑定、目标智能体约束和版本；治理判定器产生 allow、deny 或 approval；审批管理器以条件更新完成一次性终态；任务能力管理器负责签发、过期和撤销。治理状态不暴露给模型修改，模型也不能自行扩大任务能力。

3.2 执行平面与完整中介

调度前 PEP 位于所有 Connector 汇合到任务创建的窄腰，拥有事件来源、类型、目标智能体和环境上下文；调用时 PEP 位于 MCP 上游凭据之前，拥有任务能力和模型最终选择的工具。前者不能预测任务创建后的工具选择，后者不能单独恢复来源声明权，因此两个执行点并非重复检查。

3.3 证据平面

证据平面记录契约、策略、审批、任务、能力和工具结果。拒绝或待审批事件同样写入终态证据，但不伪造不存在的任务或能力；获准路径则继续追加对象依赖。验证器既检查哈希链接，也检查状态迁移

是否缺少必须阶段。

4 跨层能力治理机制

4.1 来源-类型能力契约

事件模式验证只回答字段是否合法。E2AG 在 Connector 描述中增加来源模式集合，并把它与 Connector 可声明的事件类型绑定。契约判定先检查基本 CloudEvent 结构，再查找同时满足来源与类型的版本化契约。未找到绑定时默认拒绝，因此结构合法的 `source=webhook/attacker,type=git.push` 不能借用 Git Connector 的事件语义。契约输出包含匹配契约、策略版本和稳定原因码，成为目标策略的可信输入及证据链首项。

4.2 目标策略与审批状态

对候选目标 a ，E2AG 比较事件来源、类型、声明动作、请求工具和运行环境。任一显式允许集合不匹配即返回 deny；风险条件命中返回 approval；其余返回 allow。approval 不等价于临时放行，只有审批状态以原子条件更新从 pending 单次转移到 approved 后才允许创建任务。rejected 和 expired 均为不可逆终态，重复请求不能恢复为 pending。

4.3 任务作用域能力

目标策略允许后，系统创建任务 t 并签发

$$g = \langle \rho, id(e), id(t), id(a), id(m), U, exp, state, version \rangle. \quad (6)$$

式 (6) 中每个字段都由调用时 PEP 使用： ρ 和对象标识检查跨层绑定， U 检查工具成员关系， exp 与 $state$ 检查时效和生命周期， $version$ 保留决策依据。能力令牌只在任务运行时可见，持久层保存摘要；任务完成、失败或取消后转为 revoked，超过期限转为 expired。

4.4 双执行点授权算法

图 2 只描述单个事件的动态流程，与图 1 的静态组件关系分离。

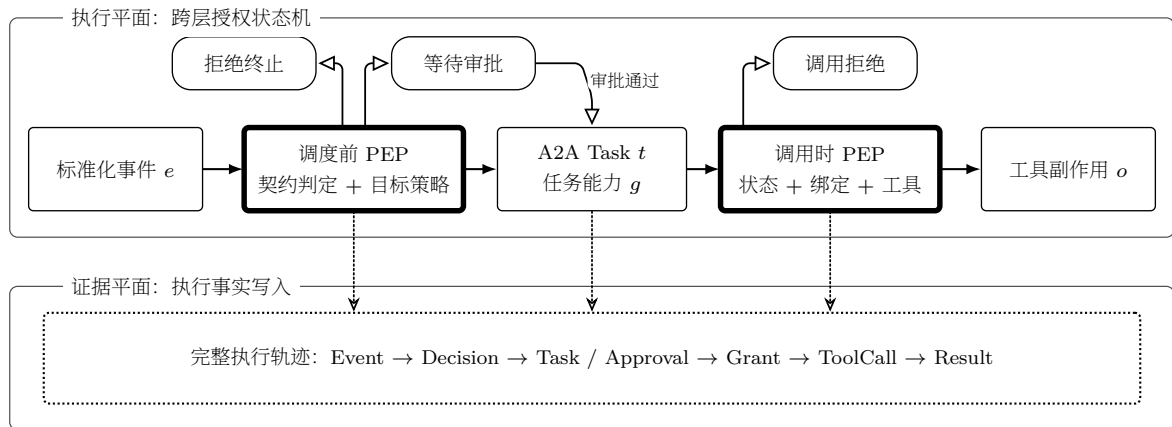


图 2: E2AG 跨层授权执行流程。两个等宽平面共享左右边界；实心箭头表示主执行路径，空心三角箭头表示治理分支，点线空心燕尾箭头表示证据写入，圆角胶囊表示暂停或终止状态。

算法 1 将式 (2) 实现为两个不可绕过的过程。事件接入过程先验证契约, 再对每个目标执行策略和审批状态转换; 只有准入成功才创建任务和能力。调用过程在接触上游凭据前检查状态、期限、对象绑定和工具集合。两个过程均追加显式阶段结果。

算法 1 E2AG 双执行点跨层授权算法

```

1: procedure GOVERN_EVENT( $e, targets, C, P$ )
2:    $\rho \leftarrow \text{NEWTRACE}$ 
3:    $d_C \leftarrow \text{EVALUATECONTRACT}(e, C)$ ;  $\text{APPEND}(\rho, d_C)$ 
4:   if  $d_C = \text{deny}$  then
5:     return  $\emptyset$ 
6:   end if
7:    $T \leftarrow \emptyset$ 
8:   for all  $a \in targets$  do
9:      $d_a \leftarrow \text{EVALUATEPOLICY}(e, P_a)$ ;  $\text{APPEND}(\rho, d_a)$ 
10:    if  $d_a = \text{approval}$  then
11:       $d_a \leftarrow \text{CONSUMEAPPROVAL}(e, a, \rho)$ 
12:    end if
13:    if  $d_a = \text{allow}$  then
14:       $t \leftarrow \text{CREATETASK}(e, a, \rho)$ 
15:       $g \leftarrow \text{ISSUECAPABILITY}(e, t, a, m, U_a, exp)$ 
16:       $\text{APPEND}(\rho, \langle t, g \rangle)$ ;  $T \leftarrow T \cup \{t\}$ 
17:    end if
18:  end for
19:  return  $T$ 
20: end procedure
21: procedure AUTHORIZECALL( $c, e, t, g, \tau$ )
22:   if  $\neg \text{ACTIVE}(g, \tau)$  then
23:     return deny
24:   end if
25:   if  $\neg \text{BINDINGSMATCH}(g, e, t, c) \vee \text{tool}(c) \notin U(g)$  then
26:      $\text{APPEND}(\text{trace}(t), \langle c, \text{deny} \rangle)$ ; return deny
27:   end if
28:    $r \leftarrow \text{FORWARDTOUPSTREAM}(c)$ 
29:    $\text{APPEND}(\text{trace}(t), \langle c, \text{allow}, r \rangle)$ ; return  $r$ 
30: end procedure

```

4.5 执行证据与验证

每项证据 l_i 包含 sequence、stage、outcome、对象标识、前驱哈希和内容哈希。令 H 为 SHA-256, canon 为确定性 JSON 规范化, 则

$$h_i = H(\text{canon}(l_i \setminus \{h_i\})), \quad l_i.\text{previous_hash} = h_{i-1}. \quad (7)$$

验证器先检查式 (7), 再根据 stage/outcome 与对象标识检查式 (2) 所需的前驱阶段。哈希链检测已取得记录的修改、换序、插入和中间删除; 没有外部 head/count 锚点时, 它不能检测整条链的尾截断。

4.6 安全性与复杂度分析

命题 1 (双执行点完整中介)：若 (A1) 所有事件驱动任务只能由 GovernEvent 创建, (A2) 所有产生外部副作用的 MCP 调用只能由 AuthorizeCall 转发, (A3) 治理状态不能被不可信输入方直接修改, 则算法 1 保持式 (5)。

证明概要. 设调用 c 已产生外部副作用。由 A2, 它必经过 AuthorizeCall 且未在状态、期限、绑定或工具成员检查处返回 deny, 因此存在满足 ValidGrant 且 $tool(c) \in U(g)$ 的 g 。能力只由 GovernEvent 在任务创建分支签发; 由 A1 和算法第 2-14 行, 该分支只能在 $B_C(e)$ 成立且策略允许或审批已原子批准时执行, 故 $Admit(e, a)$ 成立, 并建立 $e \prec t \prec g$ 。AuthorizeCall 接收同一绑定对象并追加调用, 得到 $e \prec t \prec g \prec c$, 从而式 (5) 成立。若移除任一假设或 PEP, 可分别构造来源绑定不满足或任务后集合外请求的反例; 第 6 节以配对执行观测这些反例的上游可达性。□

设契约数为 $|C|$ 、策略维度数为 k 、允许工具集合使用哈希集合。朴素契约匹配为 $O(|C|)$, 目标策略判定为 $O(k)$, 调用时状态、绑定和工具成员检查期望为 $O(1)$, 每项证据追加为 $O(1)$ 。该分析说明调用时 PEP 不随历史 trace 长度线性扫描; 实际端到端开销仍受数据库、模型和远程工具主导, 本文不把本地微基准外推为生产性能。

5 原型系统实现

本文在一个开源事件驱动智能体操作系统原型上实现 E2AG。为保持匿名评审, 仓库、分支和部署地址在当前稿件中以角色名表示, 解除匿名后公开。原型已有 Connector、CloudEvents 标准化、订阅匹配、事件调度、A2A Task 和 MCP 配置管理。E2AG 增加契约与策略判定器、调度前 PEP、审批管理器、任务能力管理器、MCP 调用 PEP 和执行证据验证器。

调度前 PEP 位于所有 Connector 汇合到 A2ATask 创建的统一 dispatcher。EventLog 保存事件及契约判定, A2ATask 通过 context 和 trace 与事件关联, ToolGrant 保存式 (6) 的能力摘要和生命周期状态。MCP PEP 代理 remote streamable HTTP: 它先验证任务能力, 再转发获准调用; `tools/list` 结果按任务允许集合裁剪, 避免向任务暴露无权调用的工具。

实现将纯判定与 I/O 分离。纯函数负责契约和目标策略, dispatcher 负责对象创建与证据持久化, MCP PEP 负责副作用前授权。实验因此可以在同一执行路径上独立开关契约、目标策略和两个 PEP, 保持数据、模型输出和上游工具一致。

事件去重最初采用“先查询、后插入”, 并发 replay 会绕过检查。修复后增加带期限的唯一 EventDedupClaim, 数据库写入成为去重线性化点。审批使用 pending、approved、rejected、expired 四态和条件更新, 使 approve/reject 竞争至多一个成功终态。这两项实现对应状态迁移模型的单调性要求。

6 实验设计与结果分析

6.1 研究问题

实验围绕性质而不是围绕脚本数量组织:

- **RQ1**: 来源契约和目标策略是否分别提供独立安全收益, 组合后能否保持正常事件可用性? 对应 I1;
- **RQ2**: 两个执行点是否具有不可替代的上下文, 完整方法能否阻止未授权副作用到达真实 HTTP 上游? 对应 I2-I3;
- **RQ3**: 执行证据能否定位首个治理失败阶段, 去重与审批状态在并发下是否保持单调性? 对应 I4 和式 (2)。

6.2 工作负载、规模与实验边界

表 2 区分独立规模和重复次数。正式有效性结论来自 60 例治理规则冻结矩阵；三名未参与语料构造的复核者通过盲表检查其规则标签，复核不增加工作负载规模。700 次固定种子变异仅作为机制压力覆盖，其中序列化去重后有 319 个不同用例。确定性端到端重复用于证明状态稳定和配对因果，不增加工作负载多样性。本研究早期已冻结一个模型在三个固定场景中的 30 个工具选择结果，本版只复用这些不可变结果进行配对回放，不新增模型调用，也不由模型构造治理回归向量。另在自有设施上以公开测试仓库的真实推送构造允许路径正对照和固定集合外请求负对照，检查 ToolGrant 与运行时 PEP 的一致性；该配对轨迹不计入统计样本。

表 2: 当前实验的独立维度与操作规模

维度	当前覆盖	证据边界
事件与规则情形	7 类来源；30 正常 +30 违规；7 类变异算子	三人盲表复核；变异集为合成压力数据
标签复核	3 名复核者 ×60 例	检查规则可复核性，不证明分布代表性
治理配置	Contract×Policy 4 种；双 PEP 4 种	内部消融，不等价于外部方法复现
外部策略引擎	OPA v1.17.0；8 个冻结 <code>tools/call</code> 用例	工具边界功能比较，不报告本地时延排名
端到端路径	480 条确定性路径；90 条模型决策配对路径	重复验证稳定性，不代表独立样本规模
模型与提示	1 个模型；每场景 1 个固定提示	不估计开放环境中的模型违规选择概率
外部端点	loopback MCP canary；1 组公开仓库-自有设施配对回归	验证部署闭合与 <code>grant-PEP</code> 一致性，不构成统计工作负载
状态与并发	SQLite；8/32 并发；每配置 100 轮	不外推其他数据库后端

正式冻结矩阵包含 GitHub、GitLab、Gitea、IMAP、通用 Webhook、Manual 和 Cron。30 个违规样本由来源-类型绑定不一致、目标来源/类型/动作/工具集合外请求和需审批事件组成；所有样本满足基本 CloudEvent 结构，以避免把解析失败计为跨层治理收益。三名复核者仅接收移除作者标签的事件字段与目标治理声明，分别标注事件类别、首个治理层和完整 E2AG 预期决策。报告违规拒绝率、正常通过率、Wilson 95% 置信区间以及多复核者 Fleiss κ 。

6.3 基准方法与公平性

表 3 给出基准。内部配置共享代码路径、事件、模型决策和上游工具，仅切换治理位置。ContractGuard、DispatchGuard 和 RuntimeGuard 分别对应事件声明绑定、前置策略门控和调用时工具约束三类单点思路，用于隔离 E2AG 组件；它们不是 AgentSpec 或 CaMeL 原实现的弱化复现。为增加独立实现参照，本文另以官方 OPA v1.17.0 执行通用 default-deny Rego 工具允许集策略（OPA-Tool）[19]。该基线只接收工具调用对象，因此仅与 RuntimeGuard 在相同的冻结工具调用切片上比较判定结果，不比较本地时延，也不把通用策略引擎等同于事件驱动 Agent 治理系统。其他外部方法仍只在表 1 按保护对象比较，不对异构公开结果排序。

6.4 RQ1: 跨层任务准入

表 4 显示 ContractGuard 只阻断来源-类型绑定违规，违规拒绝率置信区间为 [19.23%, 51.22%]；当来源具有合法契约但目标智能体不允许其动作或工具时，契约不产生作用。DispatchGuard 阻断目标能力不匹配并将需审批动作转入审批，区间为 [48.78%, 80.77%]，但无法识别来源绑定不匹配。完整组

表 3: 实验基准方法与治理能力

编号	名称	开启机制	隔离对象
C0P0 / G0R0	NoGuard	无跨层治理	无治理参照
C1P0	ContractGuard	来源-类型契约	事件声明权
C0P1 / G1R0	DispatchGuard	目标策略/调度前 PEP	任务创建权
G0R1	RuntimeGuard	任务工具允许集合	工具副作用权
OPA-T	OPA-Tool	OPA/Rego 工具允许集合	工具调用对象
C1P1 / G1R1	E2AG	准入与运行时能力	完整方法

表 4: 来源契约与目标策略的完整消融

配置	拒绝违规	违规拒绝率	正常通过	正常通过率
C0P0	0/30	0.00%	30/30	100.00%
C1P0	10/30	33.33%	30/30	100.00%
C0P1	20/30	66.67%	30/30	100.00%
C1P1	30/30	100.00%	30/30	100.00%

合覆盖两类互不重叠的拒绝原因，区间为 [88.65%, 100%]。四组正常通过率相同，说明差异来自治理对象而不是普遍收紧接入。

三名复核者在事件类别、首个治理层和完整 E2AG 预期决策三项上均达到 60/60 逐例全体一致，并与冻结作者标签完全一致；三项 Fleiss κ 均为 1.00。三份表的冻结输入字段均未被修改，且逐例备注在任意两名复核者间没有完全相同项。该结果支持规则标签明确且可由非构造者复核，不说明矩阵覆盖自然流量中的违规分布。

固定种子压力集对 7 种变异算子各执行 100 次。NoGuard、ContractGuard 和完整 E2AG 的精确决策率分别为 0、57.14% 和 100%。ContractGuard 只覆盖必填字段、规范版本、来源-类型交叉和未知类型 4 类变异，完整方法继续覆盖需审批动作与目标能力不匹配。该结果验证实现对不同变异算子的稳定分工，但由于样本由规则生成，不用于估计真实分布中的检出率。

6.5 RQ2: 双执行点与上游副作用

6.5.1 独立策略引擎的工具边界基线

首先在 `tool_call_cases.jsonl` 中选取 8 个 `tools/call` 请求，其中 4 个工具名属于任务允许集合，4 个在集合外。OPA-Tool 与 RuntimeGuard 接收相同的 3 项工具模式；Rego 策略只规定“方法为 `tools/call` 且工具名匹配允许模式时通过”，不调用 E2AG 判定代码。原冻结集另有 2 个非 `tools/call` 协议方法，它们检验 MCP 方法中介而非工具授权，故不计入 OPA-Tool 分母。

表 5: 冻结工具调用切片上的独立策略引擎基线

方法	集合内通过	集合外拒绝	判定一致
NoGuard	4/4	0/4	4/8
RuntimeGuard	4/4	4/4	8/8
OPA-Tool v1.17.0	4/4	4/4	8/8

表 5 表明任务级工具允许集的判定语义可由独立通用策略引擎复现，RuntimeGuard 的结果不是由

内部测试桩特有行为造成。该结果不表示 E2AG 在工具名判定上优于 OPA；二者在此切片功能等价。差异在保护范围：OPA-Tool 没有接收事件来源、任务准入、能力终态或跨层证据，因而不能回答同一工具调用是否继承自合法事件和有效任务。实验记录固定 OPA 版本、发布件 SHA-256、Rego 策略哈希、输入哈希和逐例结果，不报告单机进程启动时延。

确定性实验包含正常授权、来源不匹配、任务后集合外请求和需审批动作，在 4 种配置下各重复 30 次，共 480 条 dispatcher-SQLite-A2ATask-ToolGrant-MCP PEP 路径。表 6 报告真实上游可达性。

表 6: 确定性端到端执行中的上游副作用

配置	正常到达	来源不匹配	集合外请求	需审批动作
G0R0	30/30	30/30	30/30	30/30
G1R0	30/30	0/30	30/30	0/30 (审批)
G0R1	30/30	30/30	0/30	0/30
G1R1	30/30	0/30	0/30	0/30 (审批)

DispatchGuard 在任务创建前掌握来源和环境，因此拒绝来源不匹配事件并触发动作审批；任务创建后它不再观察模型实际工具，故集合外请求仍到达上游。RuntimeGuard 检查任务能力中的工具集合，阻断全部集合外请求，却不能判定一个获准工具是否由来源不匹配事件触发。完整 E2AG 保留正常副作用并阻断三类未授权副作用。该配对反例与命题 1 的两个完整中介假设一一对应。

既有模型决策回放固定正常、任务后集合外请求和来源不匹配 3 个合成事件提示，每个场景含 10 个已经冻结的工具选择结果，共 30 个。每个不可变结果在 G1R0、G0R1 和 G1R1 中复用，形成 90 条配对执行路径；本版不重新调用模型，上游为真实 loopback HTTP MCP canary。

表 7: 既有模型决策的配对执行路径

配置	正常上游到达	集合外请求副作用	来源不匹配副作用
G1R0	10/10	10/10	0/10
G0R1	10/10	0/10	10/10
G1R1	10/10	0/10	0/10

模型在集合外请求场景中均返回不在 grant 内的工具选择。为避免工具名称承载额外语义，本文仅按 grant 集合成员关系判定；结果文件保留原始标识供复核。表 7 与确定性结果一致：G1R0 不能阻止任务后的集合外请求，G0R1 不能识别工具合法但来源不匹配的事件，G1R1 同时阻断两类副作用。该实验验证模型调用进入真实 MCP PEP 后的系统行为，不把工具选择比例解释为开放环境中的模型违规选择概率。

6.5.2 自有设施上的治理一致性与集合外请求回归

为排除全部证据仅来自进程内生成器的疑问，本文在一台自有设施上另行部署仅绑定回环端口、使用独立数据库和工作区的 E2AG 实例，并在自有工具端点上执行配对回归。正对照沿用公开测试仓库的真实 GitHub push，验证 Connector、任务容器、ToolGrant、MCP PEP 与外部工具可以闭合。负对照使用该仓库的另一条真实事件，但集合外请求的工具名与参数由固定脚本直接构造，不由模型生成或选择；后续复验也只由确定性测试 harness 触发。

该配对结果保留了真实公开仓库-自有设施-任务运行时-PEP-工具端点路径，同时把负向判断收缩为可复核的治理一致性性质：集合内调用能够到达上游，集合外调用由 PEP 本地拒绝且不改变上游状

表 8: 公开仓库-自有设施治理一致性配对回归

阶段	观测结果
允许路径正对照	匿名化提交摘要; 任务 <code>completed</code> ; 5 次获准调用抵达自有工具端点; <code>grant</code> 终态 <code>revoked</code>
固定集合外请求负对照	<code>grant</code> 仅含 4 个评审工具; 集合外请求返回 403 与 <code>MCP_TOOL_NOT_GRANTED</code>
未触达上游	拒绝审计的 <code>upstream_status=null</code> ; canary 仍为 <code>effective=0</code> , 记录数不变
因果与完整性	负对照 11 项审计链校验有效, 包含 <code>grant</code> 、 <code>tool_call/deny</code> 、 <code>revoke</code> 与任务完成

态。负对照完全由固定 harness 定义, 模型不参与其构造或调度; 该结果也不被解释为模型安全率、自然流量违规概率、吞吐量或外部方法比较。

6.6 RQ3: 证据定位与并发状态

对契约拒绝、策略拒绝、审批过期、任务能力过期和 MCP 工具拒绝各注入 20 次。表 9 显示 100 条 trace 均保持统一标识、包含到达终态所需阶段, 并把首个治理失败定位到注入位置。原因在于验证器使用显式 stage/outcome 和对象依赖, 而不是事后从文本日志猜测根因。

表 9: 故障注入的治理阶段定位与证据验证

故障阶段	轨迹数	正确定位	链有效	阶段完整
契约拒绝	20	20	20	20
目标策略拒绝	20	20	20	20
审批过期	20	20	20	20
任务能力过期	20	20	20	20
MCP 工具拒绝	20	20	20	20

对内容修改、换序、trace 替换、前驱哈希替换、插入和中间删除的链内篡改, 验证器均拒绝; 没有外部锚点时尾截断仍不可检测, 符合式 (7) 的能力边界。

并发实验首先发现原有先查后插去重在 8 并发的 100 轮中均产生重复 EventLog, 共创建 588 条记录。加入唯一去重声明后, 表 10 的四组实验均满足单日志或单终态不变量。该结果证明安全性质还依赖数据库状态转移, 而不只是纯判定函数。

表 10: 修复后的 SQLite 并发状态不变量

场景	并发	请求数	成功创建/终态	不变量违规
event replay	8	800	100	0
event replay	32	3200	100	0
approve/reject	8	800	100	0
approve/reject	32	3200	100	0

7 讨论与有效性威胁

方法边界。E2AG 保证调用不超出获准来源、目标和工具集合，不判定自然语言意图类别。若请求使用已授权工具和合法工具名，只在参数中携带不符合约束的数据，当前工具粒度能力无法区分；数据流能力或参数谓词可作为正交扩展。

内部有效性。冻结矩阵按治理层构造，适合验证机制分工，不用于估计自然流量中的违规发生率。三名非构造者的盲表复核排除了明显的标签歧义，但因复核者依据同一显式治理规则判定，完全一致不能证明语料具有自然流量代表性。既有模型结果采用固定提示和工具选择约束，保证相同决策可在不同治理配置下配对复用；它证明副作用可达性，不证明开放环境中的模型违规选择概率。OPA-Tool 提供独立策略引擎的同输入功能参照，但 Rego 策略由本文按公开接口适配，只覆盖工具边界，不能替代 AgentSpec、OAP、ToolGuardian 或 Agent libOS 的原系统复现。

外部有效性与规模。当前实现限于一个 AIOS 原型、一个模型、一个合成 HTTP MCP canary、一组公开仓库-自有设施配对回归和 SQLite。该回归证明真实 GitHub payload、任务运行时与外部 MCP 服务能够闭合，并为 grant-PEP 一致性提供一次现场证据，但单仓库、单组配对和共享上游状态不能代表自然工作负载。七类来源与压力变异扩大了输入机制覆盖，却不能替代多仓库 Connector 轨迹、更多工具类型或第二 AIOS。后续实验应首先扩展这些独立维度，而不是增加相同确定性用例的重复次数。

完整中介假设。命题 1 依赖所有任务创建和外部副作用经过两个 PEP。当前原型验证 remote streamable HTTP 的 task-mode MCP 路径；stdio、SSE、Skill、内嵌服务和绕过 MCP 的本地调用尚未纳入主张。移植到其他 AIOS 时，必须重新识别任务创建和副作用调用的系统窄腰。

证据强度。哈希链保证已取得记录的内容和顺序，不能单独证明数据库未被整体回滚或截断。外部透明日志或周期性 Merkle 根锚定可以强化 I4，但不改变 I1-I3 的执行授权语义。

8 总结

本文针对事件驱动 AIOS 中事件来源、任务创建和工具副作用跨越不同信任域的问题，提出跨层能力治理方法 E2AG。该方法用来源-类型契约建立事件声明权，用目标策略和审批控制任务创建，用任务作用域能力与调用时 PEP 约束模型最终选择的工具，并通过统一执行证据连接各阶段对象。状态迁移模型和完整中介命题说明两个执行点为何缺一不可；消融、端到端配对执行、既有模型决策回放、故障注入与并发实验分别验证任务来源闭包、副作用安全、证据连续性和状态单调性。E2AG 的核心价值不是判定模型语言层输入类别，而是把非确定性模型决策置于可执行、可审计且对象绑定的系统授权边界内。

参考文献

- [1] Mei K, Zhu X, Xu W, et al. AIOS: LLM Agent Operating System. arXiv:2403.16971, 2024.
- [2] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024, 18: 186345. [doi: 10.1007/s11704-024-40231-1]
- [3] Huang HY, Li SL, Lan TW, et al. A survey on the safety of large language model: Classification, evaluation, attribution, mitigation and prospect. *CAAI Transactions on Intelligent Systems*, 2025, 20(1): 2–32 (in Chinese with English abstract). [doi: 10.11992/tis.202401006]
- [4] Mu YY, Chen HX, Li HW. Advances in security and privacy-preserving techniques for large language models. *Journal of Cybersecurity*, 2024, 2(1): 40–49 (in Chinese with English abstract). [doi: 10.20172/j.issn.2097-3136.240103]

- [5] Zhang X, Li CZ, Xu N, Zhang LT. Security challenges and response mechanisms for trustworthy large language model agents. *Information and Communications Technology and Policy*, 2025, 51(1): 33–37 (in Chinese with English abstract). [doi: 10.12267/j.issn.2096-5931.2025.01.005]
- [6] Greshake K, Abdelnabi S, Mishra S, et al. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *arXiv:2302.12173*, 2023.
- [7] Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: *Proc. of the Findings of ACL 2024*. Bangkok: Association for Computational Linguistics, 2024. 10471–10506. [doi: 10.18653/v1/2024.findings-acl.624]
- [8] Debenedetti E, Zhang J, Balunović M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Advances in Neural Information Processing Systems*, 2024, 37: 82895–82920. [doi: 10.52202/079017-2636]
- [9] Ruan Y, Dong H, Wang A, et al. Identifying the risks of LM agents with an LM-emulated sandbox. In: *Proc. of the 12th Int’l Conf. on Learning Representations*. Vienna: OpenReview, 2024.
- [10] Wang H, Poskitt CM, Sun J. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In: *Proc. of the 48th IEEE/ACM Int’l Conf. on Software Engineering*. New York: ACM, 2026. 12 pages. [doi: 10.1145/3744916.3764546]
- [11] Debenedetti E, Shumailov I, Fan T, et al. Defeating Prompt Injections by Design. *arXiv:2503.18813*, 2025.
- [12] Uchibeke U. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents. *arXiv:2603.20953*, 2026.
- [13] Ravindran A, Deochake S. ToolGuardian: Declarative Security for AI Agent-Tool Interactions. *arXiv:2607.21835*, 2026.
- [14] El Helou M, Ryder B, Troiani C, et al. Hybrid Inspection and Task-Based Access Control in Zero-Trust Agentic AI. *arXiv:2605.02682*, 2026.
- [15] Zhang YQ. Agent libOS: A Runtime Substrate for Capability-Controlled Self-Evolving LLM Agents. *arXiv:2606.03895v2*, 2026.
- [16] Cloud Native Computing Foundation. CloudEvents Specification, Version 1.0.2, 2022. <https://github.com/cloudevents/spec/tree/ce@v1.0.2>. [2026-08-14]
- [17] A2A Protocol Working Group. Agent2Agent Protocol Specification. <https://a2a-protocol.org/latest/>. [2026-08-14]
- [18] Model Context Protocol Contributors. Model Context Protocol Specification, Revision 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/>. [2026-08-14]
- [19] Open Policy Agent. OPA Documentation and Release v1.17.0: Policy Decision and Enforcement. <https://www.openpolicyagent.org/docs>; <https://github.com/open-policy-agent/opa/releases/tag/v1.17.0>. [2026-08-16]
- [20] Anderson JP. Computer Security Technology Planning Study. Technical Report ESD-TR-73-51, 1972.
- [21] Saltzer JH, Schroeder MD. The protection of information in computer systems. *Proc. of the IEEE*, 1975, 63(9): 1278–1308. [doi: 10.1109/PROC.1975.9939]
- [22] Dennis JB, Van Horn EC. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 1966, 9(3): 143–155. [doi: 10.1145/365230.365252]
- [23] Birgisson A, Politz JG, Erlingsson Ú, et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In: *Proc. of the Network and Distributed System Security Symp.* San Diego: Internet Society, 2014.
- [24] Xu JL, Wang SL, Li LM, et al. Formal verification of capability-based access control in operating system kernel. *Journal of Software*, 2025, 36(8): 3570–3586 (in Chinese with English abstract). [doi: 10.13328/j.cnki.jos.007351]
- [25] W3C. Trace Context. W3C Recommendation, 2021. <https://www.w3.org/TR/trace-context/>.

- [26] Haber S, Stornetta WS. How to time-stamp a digital document. *Journal of Cryptology*, 1991, 3(2): 99–111. [doi: 10.1007/BF00196791]
- [27] Torres-Arias S, Afzali H, Kuppusamy TK, et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In: *Proc. of the 28th USENIX Security Symp.* Santa Clara: USENIX Association, 2019. 1393–1410.